Tarea: Modelos de Ensamblaje

Ciencia de Datos — Universidad Católica del Norte

Objetivos de esta tarea

- Calcular a mano la impureza de Gini de una región y la ganancia de una división por umbral.
- Implementar en `numpy` un árbol de decisión completo: búsqueda de la mejor división, recursión y condiciones de parada.
- Construir un ensamble por remuestreo bootstrap con voto de mayoría, y observar cómo cae la dispersión del error al aumentar la cantidad de árboles.
- Agregar el sorteo de predictores por división que convierte al bagging en un bosque aleatorio, y comparar los dos sobre los mismos datos.
- Estimar la generalización con el error fuera de bolsa y contrastarlo con un conjunto reservado.
- Implementar AdaBoost sobre tocones de decisión: pesos, error ponderado, el α de cada ronda y el voto final ponderado.

Contexto

Un árbol de decisión profundo es un modelo con **sesgo bajo y varianza alta**: si se le deja crecer sin límite, clasifica sin errores los datos con que se entrenó, pero basta cambiar unas pocas observaciones para que el algoritmo voraz elija otra división en la raíz y toda la frontera se reorganice. Esa sensibilidad es un defecto cuando el árbol trabaja solo.

La idea que organiza esta tarea es que ese mismo defecto se puede convertir en materia prima. Si conseguimos B árboles *distintos* sobre el mismo problema y promediamos sus respuestas, los errores idiosincráticos de cada uno tienden a cancelarse y queda la señal que todos comparten. Toda la tarea es la implementación, a mano y desde cero, de tres maneras de conseguir esa diversidad: remuestrear los datos (bagging), remuestrear además los predictores (bosque aleatorio) y reponderar los datos según lo que el ensamble todavía no resuelve (AdaBoost).

Impureza y ganancia

Un árbol de clasificación parte el espacio con preguntas de la forma $x_j \le t$. Para elegir cuál pregunta hacer necesita una medida de cuán **mezclada** está una región. Usaremos el índice de Gini: si p_k es la proporción de la clase k dentro de la región R,

$$\mathrm{Gini}(R) = 1 - \sum_{k} p_k^2 .$$

Vale 0 cuando la región es pura y llega a su máximo, $0{,}5$ en el caso binario, cuando las dos clases están empatadas.

Una división parte R en R_1 (las n_1 observaciones con $x_j \le t$) y R_2 (las n_2 restantes). Su **ganancia** es lo que baja la impureza al pasar de la región entera al promedio ponderado de las dos mitades:

$$\text{ganancia} = \mathrm{Gini}(R) - \frac{n_1}{n}\,\mathrm{Gini}(R_1) - \frac{n_2}{n}\,\mathrm{Gini}(R_2).$$

Como umbrales candidatos basta con los puntos medios entre valores consecutivos distintos de esa columna: dos umbrales que dejan las mismas observaciones a cada lado producen exactamente la misma división.

Un ejemplo trabajado a mano

Estas son las diez observaciones de `datos_ejemplo()`, con dos predictores:

x_1	1	2	3	4	5	6	7	8	9	10
x_2	0	1	0	1	0	1	0	1	0	1
y	$+$	$+$	$+$	$+$	$-$	$+$	$-$	$-$	$+$	$-$

Hay seis positivas y cuatro negativas, así que la impureza de partida es

$$\mathrm{Gini}(R) = 1 - 0{,}6^2 - 0{,}4^2 = 0{,}48.$$

Probemos la división $x_1 \le 4{,}5$. A la izquierda quedan las cuatro primeras observaciones, todas positivas: $\mathrm{Gini}(R_1) = 0$. A la derecha quedan seis, dos positivas y cuatro negativas:

$$\mathrm{Gini}(R_2) = 1 - \left(\tfrac{2}{6}\right)^2 - \left(\tfrac{4}{6}\right)^2 = 1 - \tfrac{1}{9} - \tfrac{4}{9} = \tfrac{4}{9} \approx 0{,}4444.$$

La ganancia es entonces

$$0{,}48 - \tfrac{4}{10}\cdot 0 - \tfrac{6}{10}\cdot\tfrac{4}{9} = 0{,}48 - 0{,}2667 = \mathbf{0{,}2133}.$$

Conviene comparar con otro corte. Con $x_1 \le 6{,}5$ la izquierda tiene cinco positivas y una negativa ($\mathrm{Gini} = 10/36 \approx 0{,}2778$) y la derecha una positiva y tres negativas ($\mathrm{Gini} = 6/16 = 0{,}375$), de modo que la ganancia baja a $0{,}48 - 0{,}3167 = 0{,}1633$. Y el predictor x_2, que alterna 0 y 1, deja 3 positivas y 2 negativas a cada lado: las dos mitades tienen impureza $0{,}48$, igual que la región entera, y la ganancia es exactamente 0. Recorriendo los nueve umbrales de x_1 y el único de x_2 se comprueba que $x_1 \le 4{,}5$ es la mejor división de todas. Ese es el resultado que debe devolver tu Parte (b).

Bootstrap, bagging y bosques

Una **muestra bootstrap** de un conjunto de n observaciones se obtiene sorteando n índices con reemplazo: algunas observaciones salen repetidas y otras quedan fuera. La probabilidad de que una observación cualquiera quede fuera es $(1 - 1/n)^n$, que tiende a $e^{-1} \approx 0{,}368$. Cada árbol ve entonces cerca de dos tercios de los datos, y sobra un tercio con el que nadie lo entrenó.

Bagging entrena un árbol por muestra bootstrap y decide por voto de mayoría. Su límite es conocido: si B modelos tienen varianza σ^2 y correlación a pares ρ, el promedio tiene varianza

$$\rho\,\sigma^2 + \frac{(1-\rho)\,\sigma^2}{B}.$$

El segundo término se va a cero al agregar árboles, pero el primero no depende de B: por más árboles que se promedien, la varianza tiene un piso en $\rho\sigma^2$. Y ρ es alta justamente cuando un predictor domina a los demás, porque entonces casi todos los árboles cortan por él en la raíz y terminan pareciéndose.

El **bosque aleatorio** ataca ese piso: en *cada* división de *cada* árbol sortea m de los J predictores y obliga a elegir solo entre esos. El predictor dominante está disponible apenas en una fracción m/J de las divisiones, así que los árboles dejan de arrancar todos igual y ρ baja. Los dos extremos son ilustrativos: con $m = J$ vuelve el bagging, y con $m = 1$ el predictor de cada corte sale sorteado, lo que descorrelaciona al máximo a costa de árboles mucho peores. La elección de m es una compensación, y hay que resolverla mirando los datos.

El error fuera de bolsa

Para cada observación i existe un subconjunto de árboles que nunca la vio, porque no salió sorteada en sus muestras bootstrap. Si se la hace votar solo entre esos árboles, la predicción es honesta: para ellos, i es un dato nuevo. El promedio de esos aciertos sobre todas las observaciones es el **error fuera de bolsa** (OOB), una estimación de generalización que no exige apartar datos ni volver a entrenar nada.

AdaBoost

Bagging y bosque construyen sus modelos *en paralelo* y los tratan como iguales. El refuerzo hace lo contrario: construye los modelos *en serie*, y cada uno se especializa en lo que los anteriores no resolvieron.

AdaBoost lleva un vector de pesos sobre las observaciones, que parte uniforme en $1/n$. En la ronda i ajusta un clasificador simple $T^{(i)}$ a los datos ponderados, mide su **error ponderado**

$$\varepsilon_i = \sum_{n\,:\,T^{(i)}(x_n)\neq y_n} w_n,$$

y le asigna un peso de voto que sale de minimizar la pérdida exponencial:

$$\boxed{\alpha_i = \frac{1}{2}\,\ln\!\left(\frac{1-\varepsilon_i}{\varepsilon_i}\right)}$$

Un clasificador que falla poco recibe un α grande; uno que apenas supera al azar (ε_i cerca de $0{,}5$) recibe un α cerca de cero, y uno con $\varepsilon_i \geq 0{,}5$ no aporta nada. Después se reponderan las observaciones,

$$w_n \leftarrow \frac{w_n\, e^{-\alpha_i\, y_n\, T^{(i)}(x_n)}}{Z},$$

con Z tal que los pesos vuelvan a sumar uno. El exponente es negativo cuando el ejemplo quedó bien clasificado (su peso se encoge) y positivo cuando quedó mal (su peso engorda). La predicción final es el signo del voto ponderado, $\operatorname{sign}\big(\sum_i \alpha_i\, T^{(i)}(x)\big)$.

El clasificador simple de esta tarea es el **tocón de decisión**: un árbol de una sola división, que compara un predictor con un umbral y responde $+1$ de un lado y -1 del otro. Es deliberadamente débil, y esa es la gracia.

Archivos

- `submission.py`: el único archivo que debes modificar y el único que se entrega. Trae dos cosas: la **sección de respuestas** del comienzo, donde anotas los números de los cuatro experimentos, y las ocho funciones de la tarea con su docstring y el bloque donde va tu código.
- `util.py`: conjuntos de datos y piezas de apoyo. Ya está completo y no se toca. Ahí viven `predecir_arbol` (que fija el contrato de la estructura del árbol), `mejor_tocon` y `predecir_tocon` (el clasificador simple de AdaBoost), `umbrales_candidatos` y `clase_mayoritaria`.
- `experimento.py`: el cierre de la tarea. Corre los cuatro experimentos y deja en la terminal las cuatro tablas de las que sale la sección de respuestas. Tampoco se modifica.
- `grader.py`: corrector automático. Corre sin conexión a internet con `python grader.py`.
- `requirements.txt`: `numpy`. Nada más: no se usa ninguna biblioteca de aprendizaje automático, porque el punto de la tarea es justamente escribir estos algoritmos.

Todos los datos se generan localmente con semillas fijas: la tarea no descarga nada y dos corridas seguidas dan exactamente lo mismo.

Qué entregar

- **Entrega**: un solo archivo, `submission.py`, con las ocho funciones completadas y la sección de respuestas llena. No se entrega nada más (ni informe, ni gráficos, ni un `.zip`), y no cambies las firmas ni los nombres de las constantes, porque el corrector los llama tal como están escritos.
- Antes de entregar, ejecuta `python grader.py` en la carpeta de la tarea: debe reportar el puntaje completo de las pruebas visibles, que suman 60 de los 100 puntos. Llegar a ese total es condición *necesaria y no suficiente*, porque la nota la pone el corrector del curso, que corre esas mismas pruebas, otras sobre las mismas ocho funciones y la revisión de la sección de respuestas.
- Puntaje total: 100 puntos, repartidos en los cuatro problemas que vienen a continuación. Los 100 se corrigen solos, sin revisión a mano.
- No incluyas la carpeta `__pycache__`.

Problema 1: El árbol de decisión, desde cero [38 puntos]

Todo este problema va en `submission.py`. Las convenciones están en el docstring del archivo: `X` de forma (n, J) con una fila por observación, `y` de forma $(n,)$ con valores en $\{-1, +1\}$, y toda división escrita como `x[j] <= t`, con lo que cumple la condición en la rama izquierda.

(a) La impureza de Gini [8 puntos]

Implementa `impureza_gini(y)`, que devuelve $1 - \sum_k p_k^2$ como `float` de Python. Sobre un conjunto vacío devuelve $0{,}0$: no hay incertidumbre que medir. La función no debe suponer que las clases son dos ni que valen ± 1: depende solo de las proporciones, no de los nombres de las etiquetas ni del orden de las filas.

(b) La mejor división [13 puntos]

Implementa `mejor_division(X, y, predictores=None)`, que devuelve la tupla `(j, t, ganancia)` de la división que más reduce la impureza. Recorre los predictores candidatos y, para cada uno, los umbrales que devuelve `umbrales_candidatos`. El argumento `predictores` es la lista de columnas que se pueden mirar (`None` significa todas): es lo que más adelante hace posible el sorteo del bosque aleatorio.

Si ninguna división consigue ganancia **estrictamente positiva**, sea porque el nodo ya es puro, porque las columnas candidatas son constantes o porque ningún corte mejora nada, devuelve `(-1, 0.0, 0.0)`. Los empates se resuelven por el primero que aparece, recorriendo los predictores en el orden en que vienen y los umbrales de menor a mayor.

Pista

Sobre `datos_ejemplo()` debes obtener `(0, 4.5, 0.21333...)`, que es el desarrollo hecho a mano en el contexto. Y `mejor_division(X, y, predictores=[1])` sobre esos mismos datos debe devolver `(-1, 0.0, 0.0)`: ese predictor no informa nada. Cuidado con arrancar el máximo en $-\infty$ en vez de en 0: una división de ganancia negativa nunca se elige. Y al comparar ganancias, exige una diferencia mínima del orden de 10^{-12}: así el ruido de punto flotante no da vuelta un empate que matemáticamente no existe.

(c) El árbol completo [17 puntos]

Implementa `entrenar_arbol(X, y, profundidad_max=None, min_muestras_division=2, m=None, rng=None)`, que construye recursivamente el árbol descrito en el docstring de `predecir_arbol`. En ca-

da nodo: si corresponde detenerse, devuelve una hoja con la clase mayoritaria; si no, busca la mejor división y recurre sobre las dos mitades.

Las cuatro condiciones de parada, y basta que se cumpla una: el nodo es puro; el nodo tiene menos de `min_muestras_division` observaciones; se agotó la profundidad (`profundidad_max` llegó a 0, y `None` significa sin límite); o ninguna división da ganancia positiva.

El argumento `m` dice cuántos predictores se sortean en cada división, usando el generador `rng`. Con `m = None`, o con `m` mayor o igual que J, no hay nada que sortear y se consideran todos: ese es el árbol de siempre. Con `m` menor, se sortean `m` índices distintos y se le pasan ordenados a `mejor_division`.

Pista

La recursión sale limpia llamando a la propia `entrenar_arbol` sobre cada mitad con `profundidad_max - 1`: no hace falta un contador extra ni una función auxiliar. Arma primero el diccionario de la hoja y devuélvelo apenas se cumpla una condición de parada; la clave `"n"` guarda cuántas observaciones llegaron a esa hoja. Y pasa el *mismo* `rng` a los dos hijos: crear uno nuevo en cada nodo haría que todas las divisiones sortearan los mismos predictores.

Problema 2: Del árbol al bosque [30 puntos]

(a) Bagging, bosque aleatorio y voto de mayoría [17 puntos]

Esta parte son dos funciones.

`predecir_ensamble(arboles, X)` devuelve el voto de mayoría de una lista de árboles. Con etiquetas simétricas eso es el signo de la suma de los votos; el empate, posible cuando la cantidad de árboles es par, se resuelve a favor de $+1$, y esa misma convención rige para la lista vacía.

`entrenar_bosque(X, y, B=25, m=None, semilla=0, profundidad_max=None, min_muestras_division=2)` entrena B árboles sobre B muestras bootstrap. Con `m = None` esto es exactamente bagging; con `m` menor que J es un bosque aleatorio. Usa un único generador `rng = np.random.default_rng(semilla)` para todo, y en cada vuelta primero sortea los índices de la bolsa y después entrena el árbol pasándole ese mismo `rng`: así toda la corrida queda determinada por la semilla. Devuelve un diccionario con las claves `"arboles"`, `"bolsas"`, `"B"` y `"m"`, descritas en el docstring.

Pista

El sorteo con reemplazo es `rng.integers(0, n, size=n)`: devuelve n índices en $[0, n)$ que pueden repetirse. Si en vez de eso permutas o muestreas sin reemplazo, todos los árboles ven los mismos datos, nadie queda fuera de bolsa y la Parte (b) de este problema se vuelve imposible. El corrector revisa exactamente eso.

(b) El error fuera de bolsa [13 puntos]

Implementa `error_oob(bosque, X, y)`, que estima la generalización del ensamble sin apartar un solo dato. Para cada observación i, vota **solo** entre los árboles cuya bolsa no la contiene, compara con y_i y devuelve la fracción de desaciertos. Las observaciones que quedaron dentro de la bolsa de todos los árboles no se pueden evaluar y no entran en el promedio; si no queda ninguna evaluable, devuelve `float("nan")`.

Pista

La máscara de los que quedaron fuera de la bolsa b se arma en dos líneas: un arreglo de `True` de largo n y luego `fuera[bolsa] = False`. Acumula los votos en un arreglo de largo n, sumando solo en las posiciones que ese árbol dejó fuera, y lleva aparte una máscara de cuáles llegaron a ser evaluadas por alguien. Si tu error fuera de bolsa sale casi igual al error sobre los datos de entrenamiento, es señal de que estás votando con todos los árboles.

Problema 3: AdaBoost con tocones de decisión [17 puntos]

(a) Las rondas del refuerzo y el voto ponderado [17 puntos]

Esta parte también son dos funciones, y ninguna necesita buscar umbrales: `mejor_tocon(X, y, pesos)` de `util.py` ya devuelve el tocón de menor error ponderado, y `predecir_tocon` lo evalúa.

`entrenar_adaboost(X, y, T=10)` corre hasta T rondas. Los pesos parten uniformes en $1/n$. En cada ronda: ajusta el tocón a los datos ponderados, mide $\varepsilon = \sum_{\text{mal clasificados}} w_n$, calcula $\alpha = \frac{1}{2}\ln\big((1-\varepsilon)/\varepsilon\big)$, actualiza $w_n \leftarrow w_n\, e^{-\alpha y_n T(x_n)}$ y renormaliza para que los pesos vuelvan a sumar uno. Devuelve un diccionario con las listas `"tocones"`, `"alfas"` y `"errores"`, del mismo largo.

Dos resguardos, los dos descritos en el docstring: si $\varepsilon \geq 0{,}5$ el tocón no supera al azar y el bucle se detiene sin agregarlo, con lo que las listas pueden quedar más cortas que T; y ε se recorta al intervalo $[10^{-10},\ 1-10^{-10}]$ antes del logaritmo, para que un tocón perfecto no divida por cero.

`predecir_adaboost(modelo, X)` devuelve el signo del voto ponderado $\sum_i \alpha_i\, T^{(i)}(x)$, con $+1$ cuando el puntaje es exactamente cero y también cuando el ensamble está vacío.

Pista

Comprueba la primera ronda sobre `datos_ejemplo()`: con pesos uniformes de $1/10$, el mejor tocón es el corte $x_1 \leq 4{,}5$ que ya trabajamos, y falla en las observaciones $x_1 = 6$ y $x_1 = 9$. Entonces $\varepsilon = 0{,}2$ y $\alpha = \frac{1}{2}\ln(0{,}8/0{,}2) = \frac{1}{2}\ln 4 = \ln 2 \approx 0{,}6931$. Si te da otro número, el problema está en el error ponderado y no en el logaritmo. Y ojo con el orden: los pesos se actualizan con el α de *esta* ronda, después de calcularlo.

Problema 4: Lo que muestran los experimentos [15 puntos]

(a) La sección de respuestas de `submission.py` [15 puntos]

Con las ocho funciones completas, ejecuta

```
python experimento.py
```

Demora alrededor de medio minuto y deja en la terminal las cuatro tablas que cierran la tarea: la profundidad de un árbol solo, la caída de la dispersión del error al aumentar B, la comparación entre bagging y bosque aleatorio con su error fuera de bolsa, y las rondas de AdaBoost.

Anota esos números en la **sección de respuestas** que abre `submission.py`. Son ocho constantes que ya vienen con sus claves puestas y con un comentario que dice de qué tabla y de qué columna sale cada una. No se calculan a mano ni se copian de acá: salen de correr tu propio código.

Dos reglas de forma. Los números se comparan con **tres decimales**, así que da lo mismo anotar $0{,}1234$ o $0{,}123$. Y las claves de cada diccionario tienen que quedar exactamente como vienen, ni una más ni una menos.

Del tercer experimento se piden dos cosas: el error del árbol suelto y la fila de bagging, la de «todos (8)». Las filas con sorteo dependen del orden en que cada implementación consume el generador al recorrer las ramas, de modo que dos programas igual de correctos pueden llegar a tablas distintas y no admiten una respuesta única.

Pista

Estos números salen de tu código, así que una fila rara casi siempre es una falla de las partes anteriores y no un error de copia. Tres señales que conviene mirar antes de dar por buena la corrida:

- en el primer experimento, sin tope de profundidad el error de entrenamiento tiene que llegar a 0 y el reservado no;
- en el segundo, la desviación estándar entre las seis semillas tiene que bajar de forma monótona al aumentar B;
- en el cuarto, el error de entrenamiento baja con T mientras el reservado toca su mínimo temprano y después empeora.

Si alguna de las tres no se cumple, vuelve al código antes de anotar nada.